

Reviewer Pack — Free Cumulant Sum Gap in Read-Twice BPs for 3-SAT

Entry eeb829cc3120 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-13 06:30:49 UTC

Free Cumulant Sum Gap in Read-Twice BPs for 3-SAT

Entry ID: eeb829cc3120 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-13 06:30:49 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Read-Twice Branching Programs for 3-SAT

Statement:

For a 3-SAT instance φ with n variables, let $\rho(\varphi)$ be the sum of free cumulants of the transition matrix of its minimal read-twice BP. Then $\rho(\varphi) = O(\log n)$ if φ is satisfiable, but $\rho(\varphi) = \Omega(n)$ if φ is a hard 3-SAT instance (e.g., a random 3-CNF with clause density 4.2).

Rationale (proposer's reasoning):

Free cumulants capture non-commutative independence structures in transition matrices, which may reveal hidden symmetries in BP state transitions. Hard 3-SAT instances exhibit chaotic state correlations that resist free cumulant compression, while satisfiable instances have structured dependencies amenable to logarithmic compression.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1c93122888cfe03a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def generate_3sat_instance(n, clause_density):
    num_clauses = int(n * clause_density)
    literals = list(range(-n, 0)) + list(range(1, n+1))
    clauses = []
    for _ in range(num_clauses):
        clause = random.sample(literals, 3)
        clauses.append(clause)
    return clauses

def r_transform(matrix):
    n = len(matrix)
    if n == 0:
        return 0
    det = matrix[0][0]
    cofactor_matrix = [[matrix[i][j] for j in range(1, n)] for i in range(1, n)]
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[:i] + matrix[i+1:]]
            det += (-1) ** (i + j) * matrix[i][j] * r_transform(minor)
    return det

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 40
    clause_density = 4.2
    instance = generate_3sat_instance(n, clause_density)

    # Construct the transition matrix (simplified for demonstration)
    transition_matrix = [[0] * (2**n) for _ in range(2**n)]
    for state in range(2**n):
        for literal in range(-n, n+1):
            if literal == 0:
                continue
            next_state = state ^ (1 << abs(literal)-1)
            transition_matrix[state][next_state] += 1

    # Compute the free cumulant sum via R-transform algorithm
    free_cumulant_sum = r_transform(transition_matrix)
```

```

metric_name = "free_cumulant_sum"
metric_value = abs(free_cumulant_sum)
instances_tested = 1
conjecture_holds = metric_value <= 0.9 * math.log(n) if instance else True
counterexample = "" if conjecture_holds else f"hard_3sat_instance_with_clause_density_{clause_

return {
    "metric_name": metric_name,
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={f\"hard_3sat_instance_with_clause_density_{4.2}\"}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673e46fc.py", line 77, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_673e46fc.py", line 46, in run_trial
    transition_matrix = [[0] * (2**n) for _ in range(2**n)]
                        ~~~~~^~~~~~
MemoryError

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with MemoryError, preventing data collection. Support condition cannot be evaluated without results. | next: Optimize memory usage in BP matrix construction (e.g., sparse representations) and retry with smaller n

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109889
2	propose	ollama_remote	qwen3:8b	0	0	32339
3	preregistration	ollama_remote	qwen3:8b	0	0	30229
4	novelty	ollama_remote	qwen3:8b	0	0	22210
5	novelty	ollama_remote	qwen3:8b	0	0	14978
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15435
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9235
8	judge	ollama_remote	qwen3:8b	0	0	20114

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 254428 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/eeb829cc3120.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/eeb829cc3120.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/eeb829cc3120.tar.gz (if generated)